BEAT: Tokenizing and Generating Symbolic Music by Uniform Temporal Steps

Anonymous Authors[1]

Abstract

Tokenizing music to fit the general framework of language models is a compelling challenge, especially considering the diverse symbolic structures in which music can be represented (e.g., sequences, grids, and graphs). To date, most approaches tokenize symbolic music as sequences of musical events, such as onsets, pitches, time shifts, or compound note events. This strategy is intuitive and has proven effective in Transformer-based models, but it treats the regularity of musical time implicitly: individual tokens may span different durations, resulting in non-uniform time progression. In this paper, we instead consider whether an alternative tokenization is possible, where a uniform-length musical step (e.g., a beat) serves as the basic unit. Specifically, we encode all events within a single time step at the same pitch as one token, and group tokens explicitly by time step, which resembles a sparse encoding of a piano-roll representation. We evaluate the proposed tokenization on music continuation and accompaniment generation tasks, comparing it with mainstream event-based methods. Results show improved musical quality and structural coherence, while additional analyses confirm higher efficiency and more effective capture of long-range patterns with the proposed tokenization.

1. Introduction

As language models continue to advance, it is increasingly compelling to explore how symbolic music generation can be integrated into this paradigm. A central challenge lies in the tokenization of music, as music is a highly structured form of information that exhibits regular temporal patterns, polyphonic organization, and rich expressive variation. Music can be encoded in diverse ways, each highlighting different structural aspects: **grid-based representations**, such as piano-rolls, which discretize music into fixed time units (e.g., beats or tatums) and naturally exhibit temporal shift invariance; **notation-based representations**, such as ABC (Walshaw, 2011) or MusicXML (Good, 2001), which capture syntactic and hierarchical relationships; and **event-based representations**, such as MIDI (MIDI Manufacturers Association, 1996), which are well suited for representing temporally ordered performance actions.

Among these representations, event-based tokenizations have become dominant in symbolic music generation (Huang & Yang, 2020; Hsiao et al., 2021). Their one-dimensional, sequential structure provides a straightforward way to tokenize musical control messages and integrates naturally with Transformer-based language models. Recently, notation-based formats have also been explored (Wang et al., 2025). Both approaches represent music as chronologically ordered sequences of events; however, they do not explicitly encode the uniformity of the temporal grid, a property that is inherent to grid-based representations.

In event- or notation-based representations, each token, or the interval between successive tokens, may span a variable and uncertain duration, ranging from a fraction of a beat to multiple beats. As a result, the model must implicitly infer the underlying temporal grid, placing an additional burden on learning regular temporal structure. In contrast, human musical perception is strongly grounded in *evenly spaced temporal units*, such as beats or pulses, which serve as fundamental primitives to form higher-level music understanding (London, 2012; Huron, 2006). Motivated by this observation, we treat uniform temporal discretization as a core inductive bias in tokenization design and investigate how such representations can be naturally integrated with Transformer-based models.

In this paper, we instantiate the idea of grid-based tokenization as **BEAT** (**B**eat-wise **E**ncoding for **A**utoregressive **T**ransformers).[1] BEAT assumes a quarter note, referred to as a beat, as a fixed temporal unit. We first encode the information for each beat and each pitch into a single token. Tokens corresponding to all-rests are omitted, and the remaining tokens within the same beat are concatenated

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

[1]Demo page: https://anonymous.4open.science/w/BEAT-349F/.

to form a beat-level sequence. The full token sequence is then constructed by concatenating beat-level sequences in chronological order. The core idea of BEAT is to provide an efficient representation of the piano-roll format, avoiding nested or hierarchical designs that are difficult to scale (Wang et al., 2020; Jiang et al., 2020a), while explicitly leveraging the inherent sparsity of piano-roll representations. This design ensures that each token corresponds to a fixed temporal duration of one beat, achieves compactness comparable to event-based representations, and retains the explicit temporal regularity inherent to grid-based representations.

We train an autoregressive Transformer model using our proposed tokenization scheme and compare it with existing tokenization methods across several tasks, including piano and multi-track continuation, pattern-controlled generation, and real-time music accompaniment. Both subjective and objective evaluations indicate that our method outperforms baselines on most criteria. Further analyses highlight several key advantages of our approach:

- **Compactness**: BEAT produces more compact sequences than existing methods. It also exhibits higher compressibility under BPE, suggesting more reusable substructures that facilitate pattern learning.
- **Long-term structural coherence**: BEAT captures long-range dependencies more effectively. It achieves human-like variation and coherence across long spans, whereas existing methods tend to either introduce excessive novelty or fall into repetitive loops.
- **Real-time controllability**: BEAT provides explicit and uniform temporal representation at the beat level. This enables real-time conditioning that is difficult to achieve with event- or notation-based tokenizations.

2. Related Work

In this section, we review three areas of related work. We begin by comparing existing music tokenization approaches and highlighting the limitations of grid-based representations (Section 2.1). Next, we examine methods for incorporating grid-based structures into Transformer models (Section 2.2). Finally, we provide an overview of symbolic music language models and the generation tasks they enable (Section 2.3).

2.1. Symbolic Music Representations

Piano-roll representations treat music as a 2D time-pitch matrix, analogous to images (Briot et al., 2019). This format offers direct temporal access and explicit sustain states. Piano-rolls have been adopted in VAE-based models for representation learning (Yang et al., 2019; Wang et al., 2020) and diffusion models for score generation (Min et al., 2023; Lv et al., 2023). However, their 2D structure does not naturally fit the sequential paradigm of autoregressive Transformer language models, and severe sparsity makes direct serialization inefficient.

Event-based representations serialize music as sequences of MIDI control messages. Early work (Oore et al., 2020) tokenized raw MIDI events for performance modeling. Subsequent methods introduced metrical structure: REMI (Huang & Yang, 2020) added bar and position tokens, while Compound Word (Hsiao et al., 2021) grouped attributes into compound tokens to reduce sequence length. More recently, Nested Music Transformer (Ryu et al., 2024) addresses intra-token dependencies via a sub-decoder. Despite these advances, event-based representations primarily model the temporal ordering of control actions, overlooking other inductive biases that are fundamental to music structure.

Notation-based representations, such as ABC notation (Walshaw, 2011) and MusicXML (Good, 2001), were originally designed for human-readable scores. Some work also explores graph-based representations, which model music as nodes connected by edges encoding temporal or harmonic relationships (Jeong et al., 2019; Karystinaios & Widmer, 2022). Recent work explores notation-based representations in LLMs: ChatMusician (Yuan et al., 2024) treats ABC as a second language for LLaMA, MuPT (Qu et al., 2024) applies BPE tokenization to ABC, and NotaGen (Wang et al., 2025) uses Interleaved ABC for multi-track generation. While text-compatible, notation-based methods remain less prevalent than event-based approaches. Moreover, these formats often rely on token combinations to express information, introducing complex inter-token relationships that increase sequence length and may burden sequence modeling.

2.2. Autoregressive Modeling of Grid-Based Data

Tokenizing grid-based data structures such as piano-rolls for Transformer input presents unique challenges. In computer vision, researchers have explored various approaches to process images with Transformers. ViT (Dosovitskiy et al., 2021) partitions images into fixed-size patches and linearly embeds them as token sequences. VQ-VAE (van den Oord et al., 2017) and VQGAN (Esser et al., 2021) learn discrete codebooks to compress images into compact tokens. Building on these representations, ImageGPT (Chen et al., 2020) and LlamaGen (Sun et al., 2024) demonstrate that standard autoregressive Transformers can effectively model such discretized 2D data.

However, unlike images, piano-rolls are inherently sparse—at any given moment, only a small subset of pitches are active while the vast majority remain silent. Directly applying patch-based approaches would result in most patches being empty, while the few non-empty patches contain ex-

Table 1. Token types in BEAT encoding. The "Format" column shows the token notation used in Figure 1.

Category	Format	Meaning	Range
Pattern	PATx	pattern token $s = x$	0–80 (τ=4)
Pitch	PITx	pitch token $d = x$	0–127
Velocity	VELx	velocity token $v = x$	0–127
Rest	REST	empty beat marker	—
Instrument	INSx	instrument token $i = x$	0–128
Beat grid	BEAT, BAR	beat/bar marker	—
Others	TEMx	tempo $= x$	30–209
	TSx	time signature $= x$	varies

cessive information. Music is essentially composed of discrete events such as pitch, rhythm, and their interrelationships. The key challenge, therefore, lies in converting the grid-based piano-roll into a sparse token representation that captures these discrete musical properties.

2.3. Symbolic Music Generation Models

Recent years have witnessed significant progress in Transformer-based symbolic music generation. REMI (Huang & Yang, 2020) improved rhythmic coherence through beat-relative position encoding. Anticipatory Music Transformer (Thickstun et al., 2024) extends the autoregressive framework with an anticipation mechanism, enabling infilling and continuation. FIGARO (von Rütte et al., 2023) achieves fine-grained control over musical attributes such as density and instrumentation.

While these models have achieved remarkable success in offline tasks such as continuation, infilling, and conditional generation, their underlying tokenization schemes are inherently unsuitable for real-time accompaniment generation—a scenario with increasing importance. Real-time accompaniment requires the model to generate responses instantaneously as melodic input arrives. Yet, existing encoding methods typically require complete musical segments before tokenization, which precludes streaming processing. Current solutions for real-time accompaniment, such as SongDriver (Wang et al., 2022) and RL-Duet (Jiang et al., 2020b), rely on specialized architectures or reinforcement learning strategies. This gap between existing tokenization designs and real-time generation requirements motivates the need for a new representation that bridges this divide.

3. Methodology

This section introduces BEAT, our proposed grid-based tokenization framework. We begin by describing the encoding procedure (Section 3.1), followed by a discussion of its desirable structural properties (Section 3.2), and finally outline the autoregressive modeling approach (Section 3.3).

3.1. Beat-Wise Encoding

BEAT representation converts symbolic music with multiple tracks into a token sequence. The core idea is to treat a *beat* as the basic unit (typically aligned with a musical beat, e.g., a quarter note, though this can vary), encoding all musical events within a beat into a compact token sequence before concatenating across beats. The encoding consists of three steps: encoding each pitch's information within a beat (which we referred to as a *pattern*), assembling patterns into a beat-level sequence, and constructing the final token sequence. Figure 1 illustrates the encoding process and Table 1 summarizes the token types.

We represent symbolic music as a three-state piano-roll $X \in \{0, 1, 2\}^{P\times T}$, where P is the number of pitches and T is the number of time steps. Each entry $X[p, t]$ indicates the state of pitch p at time t: 0 for silence, 1 for onset (a note attack), and 2 for sustain (a continuation of a previous onset). All symbolic music is quantized at sixteenth-note resolution by default. We partition X into beats using a resolution parameter τ (e.g., $\tau = 4$ when a beat corresponds to a quarter note), producing $N = \lceil T/\tau \rceil$ beat matrices $B^{(1)}, B^{(2)}, \ldots, B^{(N)}$, where each $B^{(i)} \in \{0, 1, 2\}^{P\times\tau}$ represents a beat segment.

Step 1: Pitch pattern encoding. Within a beat represented by its beat segment B, the state vector for a given pitch p, denoted by $\mathbf{s}_p = B[p, :] \in \{0, 1, 2\}^\tau$, describes the temporal pattern of the pitch over τ time steps. We encode this as a pattern token s via base-3 conversion:

$$s_p = \sum_{t=0}^{\tau-1} \mathbf{s}_p[t] \cdot 3^{\tau-1-t}. \tag{1}$$

In our grid-based representation, each pattern is paired with an overall velocity descriptor. In the current implementation, we use a single value v_p to summarize the velocity of pitch p within the beat, computed as the mean of its MIDI velocities. This provides a compact yet informative representation of the dynamics, which can also be extended in the future. We denote the resulting tokens as PATx for $s_p = x$ and VELx for $v_p = x$.

Step 2: Beat-level assembly. While pattern tokens capture what happens to each pitch and each beat, we need an extra token to specify positions along the pitch axis. For each beat matrix B, we identify the set of active pitches $\mathcal{A} = \{p : \exists\, t,\ B[p, t] \neq 0\}$. Let $M = |\mathcal{A}|$ denote the number of active pitches, sorted in descending order as $p_1 > p_2 > \cdots > p_M$. We encode pitch positions as pitch tokens d using relative intervals:

$$d_1 = p_1, \quad d_j = p_{j-1} - p_j \text{ for } j \geq 2. \tag{2}$$

That is, the first pitch uses an absolute index while subsequent pitches use relative offsets. Finally, we pair each pitch

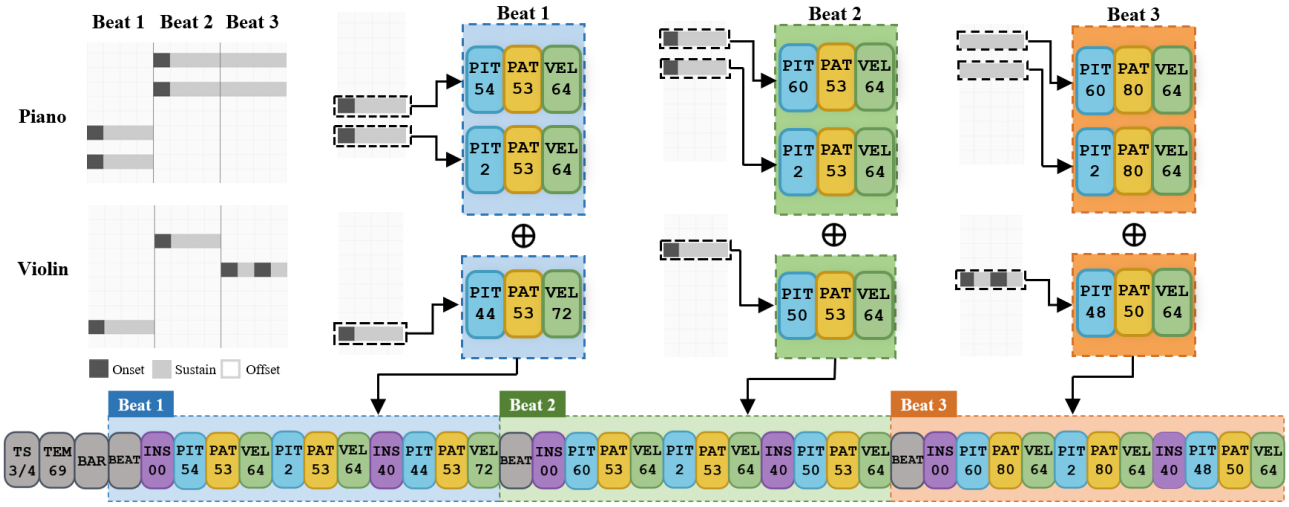


Figure 1. Overview of the BEAT encoding framework.

token with its corresponding pattern and velocity tokens, representing the beat as:

$$\mathbf{u} = (d_1, s_{p_1}, v_{p_1}) \oplus (d_2, s_{p_2}, v_{p_2}) \oplus \cdots \oplus (d_M, s_{p_M}, v_{p_M}). \tag{3}$$

For empty beats ($M = 0$), we set $\mathbf{u} = \texttt{Rest}$, a special token.

Step 3: Sequence construction. To assemble beat-level sequences $\mathbf{u}$ into a complete multi-track sequence, we introduce three additional token categories. We use *beat grid tokens* to provide temporal structure. Each beat is preceded by a `BEAT` marker that delimits beat boundaries, ensuring each beat remains a self-contained unit. `BAR` markers indicate measure boundaries.

We use *instrument tokens* to identify tracks within each beat. Each track's content is prefixed by an instrument token i_k, where k corresponds to the MIDI program number. The sequence for a complete piece takes the form:

$$\texttt{BEAT}\ (i_1\ \mathbf{u}_1^{(1)})\ (i_2\ \mathbf{u}_2^{(1)})\ \ \texttt{BEAT}\ (i_1\ \mathbf{u}_1^{(2)})\ (i_2\ \mathbf{u}_2^{(2)})\ \ldots, \tag{4}$$

where $\mathbf{u}_k^{(n)}$ denotes the (pitch, pattern, velocity) pairs for track k at beat n. Parentheses are added for readability. Crucially, instrument tokens attach to beats rather than individual notes.

Finally, the sequence is augmented with *other tokens* encoding musical attributes such as tempo and time signature at bar boundaries. Detailed encoding and decoding algorithms are provided in Appendix B.

3.2. Structural Properties of the Encoding

The BEAT tokenization exhibits several desirable structural properties:

First, beats are explicit and localized units. Each beat corresponds to a contiguous token subsequence, in which all musical events within that beat are grouped together. This introduces an inductive bias aligned with human perception of music in fixed time units (London, 2012), and also eliminates information leakage across beats—a benefit for conditioning in generation tasks (see Section 3.3).

Second, the token sequence is compact and scales with musical complexity. By leveraging the sparsity of piano-roll input, our representation scales as $O(N \cdot \bar{M})$, where N is the number of beats and $\bar{M}$ is the average polyphony. This contrasts with naïve piano-roll serialization and aligns with the intuition that longer or denser music requires longer descriptions.

Third, the representation is consistent under pitch and rhythm shifts. Transposition affects only the first pitch token d_1; all subsequent intervals remain intact, enabling generalization across keys. Likewise, the within-beat encoding of a musical figure is invariant to its temporal position, since each beat tokenization depends solely on its local content $B^{(i)}$, not on beat index i. These inductive biases reduce what must be learned from data and promote generalization over musically meaningful transformations. We empirically validate how this structure enables efficient learning of given musical patterns in Section 5.2.

3.3. Unified Autoregressive Framework

With our BEAT tokenization, music sequences can be directly modeled using standard autoregressive transformers. Given a token sequence $\mathbf{x} = (x_1, x_2, \ldots, x_L)$, the model factorizes the joint probability as $p(\mathbf{x}) = \prod_{i=1}^{L} p(x_i \mid \mathbf{x}_{<i})$ and is trained to minimize the cross-entropy loss $\mathcal{L} = -\sum_{i=1}^{L} \log p_\theta(x_i \mid \mathbf{x}_{<i})$.

Our tokenization enables versatile control in music gener-

ation by unifying various tasks under a single autoregressive framework. It requires only minor adjustments to the sequence structure rather than task-specific architectures or training procedures. As with other standard tokenizations (von Rütte et al., 2023), control is achieved by conditioning on tokens for attributes such as tempo and meter, and on past tokens for continuation. A key advantage of our beat-wise encoding is its strict causal and evenly spaced structure in musical time: all information used for prediction lies strictly in the past, with no ongoing events, and each beat is represented by at least one token, ensuring no temporal skips. This makes the framework well-suited for *real-time generation*, where the model predicts the next beat (e.g., accompaniment) based on prior context (e.g., melody and past accompaniment). We will further demonstrate this setup in Section 5.3.

4. Experiments

In this section, we evaluate the performance of our proposed BEAT tokenization. Comparing against baseline methods, we conduct quantitative evaluation on the task of *music continuation*, which aims to extend a 4-bar prompt in *piano* and *multi-track* formats, respectively. Section 4.1 presents the datasets used and the training details. Section 4.2 introduces the baseline tokenization methods. Our evaluation is divided into two parts: objective evaluation as detailed in Section 4.3, and subjective evaluation as covered in Section 4.4. In Section 4.5, we further highlight the long-term structural coherence achieved by our method via qualitative visualizations. Additionally, ablation studies are provided in Appendix A.

4.1. Datasets and Training Details

We evaluate multi-track and piano continuation using different datasets. The multi-track setting is evaluated on Lakh MIDI Dataset (LMD) (Raffel, 2016), which yields 148K pieces (∼8.6K hours) after processing. For the piano setting, we supplemented the 15K piano pieces found in LMD with 193K pieces from MuseScore, yielding 208K piano pieces in total. All data is quantized to 16th-note resolution and split 80/10/10 at song level with transposition augmentation.

Our language model is a 16-layer Transformer decoder following LLaMA (Touvron et al., 2023), comprising 150M parameters. We refer readers to Appendix C and D for more details on data processing and model training. These settings are kept identical for our tokenization and all baseline methods to maintain a fair comparison.

4.2. Baseline Tokenization Methods

We compare BEAT against four representative tokenization methods. **REMI** (Huang & Yang, 2020) is a widely adopted event-based approach that serializes notes as (position, pitch, duration, velocity) events. **REMI+** (von Rütte et al., 2023) extends REMI for multi-track music by adding instrument tokens, while **Compound Word (CPW)** (Hsiao et al., 2021) aggregates the event sequence associated with each note into a single compound token. **Interleaved ABC** (Wang et al., 2025) takes a different approach, extending ABC notation to multi-track music by interleaving tracks. Additionally, we include **Anticipatory Music Transformer (AMT)** (Thickstun et al., 2024) as an external reference using their released models: AMT-Small (128M parameters, closest to our model size) and AMT-Large (780M parameters). Implementation details of all baselines are provided in Appendix E.

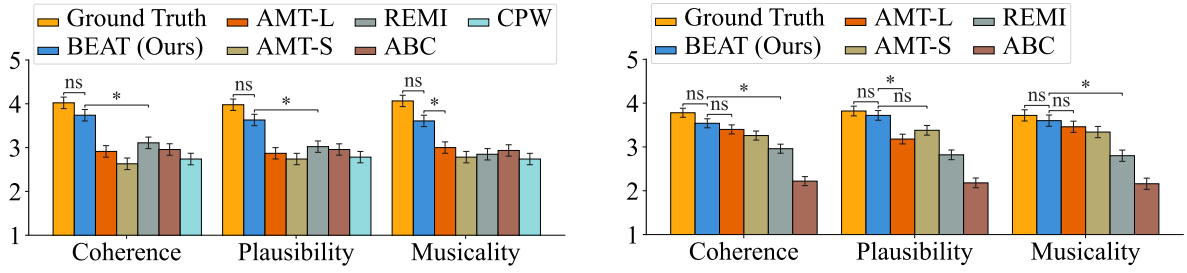
Table 2. Objective evaluation results on music continuation (lower is better for all metrics). †Released models; all others trained with identical 150M-parameter architecture. ‡We use REMI for piano and REMI+ for multi-track; they are equivalent for single-track. "—": unsupported setting.

	Piano			Multi-track		
Method	JS_{GC}↓	JS_{SC}↓	FMD↓	JS_{GC}↓	JS_{SC}↓	FMD↓
Int. ABC	.677	.023	522.4	.594	.036	580.0
REMI(+)‡	.552	.038	550.9	.313	**.008**	463.2
CPW	.634	.024	587.0	—	—	—
AMT-S†	.603	.055	447.1	.358	.078	449.3
AMT-L†	.625	.053	445.2	.353	.029	441.8
BEAT(Ours)	**.039**	**.021**	**436.7**	**.043**	.009	**420.9**

4.3. Objective Evaluation

We introduce three metrics to evaluate music continuation: **Groove Consistency (GC)** (Dong et al., 2020), **Scale Consistency (SC)** (Dong et al., 2020), and **Fréchet Music Distance (FMD)** (Retkowski et al., 2024). **GC** measures rhythmic regularity as the similarity of onset patterns between adjacent measures. **SC** (Dong et al., 2020) measures tonal coherence as the pitch-in-scale rate. Both metrics are per-piece statistics that reflect rhythmic or harmonic regularity, and we report JS divergence between generated and ground truth distributions (JS_{GC}, JS_{SC}; lower indicates greater similarity to real music). On the other hand, **FMD** (Retkowski et al., 2024) measures the latent distributional distance between generated samples and the ground truth (lower is better), which indicates more general similarity. Detailed metric definitions are provided in Appendix F.

We evaluate piano continuation and multi-track continuation separately. For each setting, we sample 20 pieces from the test set and generate 10 continuations per prompt (truncated to 30 bars), yielding 200 samples per tokenization method. Results are presented in Table 2. REMI and Compound Word do not natively support multi-track; REMI+ extends REMI with instrument tokens. BEAT achieves the best JS_{GC} and FMD on both settings, demonstrating strong rhythmic



(a) Evaluation on piano continuation.

(b) Evaluation on multi-track continuation

Figure 2. Subjective evaluation results. Bar plots report mean ratings and standard errors. * indicates a statistically significant difference ($p < 0.05$) based on pairwise t-tests with Holm-Bonferroni correction; "ns" denotes non-significant differences.

coherence and distributional similarity to real music. While some baselines achieve competitive JS_{SC}, they exhibit substantially worse JS_{GC}, indicating irregular rhythmic patterns despite reasonable tonal coherence.

4.4. Subjective Evaluation

We further conduct a double-blind listening survey to evaluate music quality. Our survey contains 5 pages each for piano and multi-track continuation settings (10 pages in total). Each page begins with a 4-bar prompt drawn from the corresponding test split, followed by continuation samples generated by our method and each baseline. The ground-truth sample is also included as a perceptual anchor. Each sample is truncated to 30 bars and synthesized to audio at 120 BPM, resulting in approximately 1 minute of audio per sample. Both the page order and the sample order within each page are randomized. We request that participants listen to each sample and evaluate its musical quality on a 5-point Likert scale from 1 to 5. The evaluation considers 3 criteria: 1) *Coherence*, measuring how well the continuation fits the prompt; 2) *Plausibility*, assessing musical well-formedness and structure; and 3) *Musicality*, the overall perceptual quality. Additional details of the subjective evaluation are provided in Appendix G.

A total of 32 participants with diverse musical backgrounds completed our survey. Figure 2 shows the mean ratings and standard errors computed using within-subject ANOVA, with significant main effects (p-value $p < 0.05$) observed across all evaluation criteria. Among the baselines, REMI performs better in the piano continuation setting (Figure 2a), while AMT achieves stronger results for multi-track music (Figure 2b), which may reflect their respective design focuses. In comparison, our method consistently surpasses each baseline in both settings, yielding the highest subjective ratings overall. Post-hoc pairwise t-tests further reveal that, for piano continuation, our method significantly outperforms all baselines across all evaluation criteria ($p < 0.05$ with Holm-Bonferroni correction). For multi-track continuation, AMT is competitive, though our method maintains marginally higher scores. While ground-truth samples are consistently preferred over generated ones, the difference between our method and the ground truth is not statistically significant. Qualitative examples and audio samples are available in Appendix H.

4.5. Repetition-Diversity Analysis

To evaluate long-term structural coherence, we analyze the balance between *repetition* and *diversity* in piano continuation samples. Natural music typically exhibits thematic repetition in balance with variation. Too much repetition can make the music feel monotonous, whereas too much variation can make it feel disjointed and hard to follow.

To assess the balance level, introduce *unique beat ratio*, a metric that quantifies the trade-off between repetition and variation. In this setting, a test piece is first converted to the piano-roll representation and segmented into beat-wise intervals. A beat interval is considered unique if it has not appeared previously. Thereby, the *unique beat ratio* at position t is defined as the cumulative unique count divided by t. Values near 1.0 indicate high diversity, while lower values reflect increasing repetition. We evaluate at two granularities (1-beat and 2-beat) and two matching criteria (onset-only and full state), analyzing 200 samples per method.

Figure 3 shows the unique beat ratio curves over 120 beats. BEAT closely tracks the ground truth distribution, achieving ratios within 0.3–1.2% of ground truth at beat 120, while other methods deviate by 5–30%. CPW exhibits excessive diversity (ratios 0.88–0.99 vs. ground truth 0.56–0.78), indicating a failure to develop coherent thematic material. Interleaved ABC shows excessive repetition (ratios 7–11% below ground truth). These results suggest that BEAT learns the natural repetition-variation balance present in real music.

5. Further Analysis

One may wonder how our model achieves superior performance despite its relatively small data scale and parameter count. In this section, we analyze the structural properties of our method and provide insights underlying its performance. Section 5.1 analyzes the tokenization compactness, which

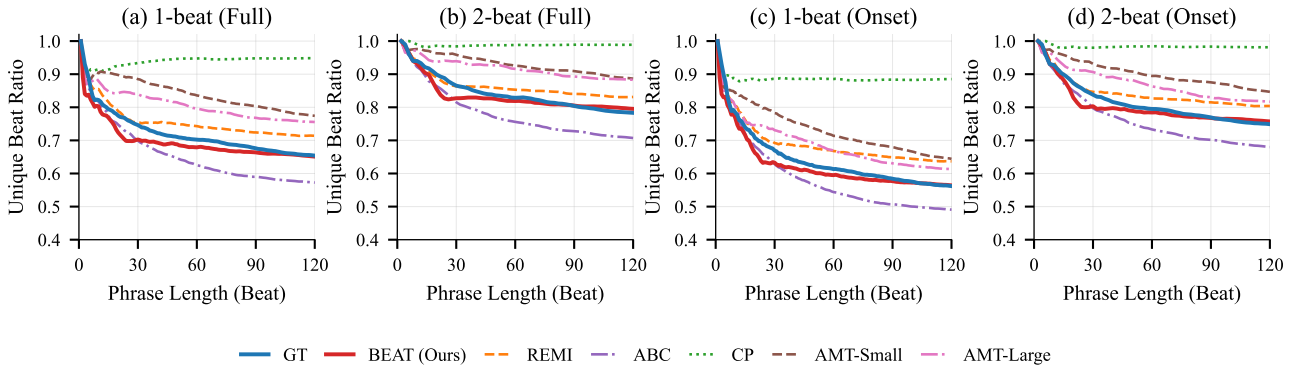


Figure 3. Unique beat growth curves for music continuation. BEAT (red) closely tracks ground truth (blue), while Compound Word (green) shows excessive diversity and Interleaved ABC (purple) shows excessive repetition.

reduces training burden. Section 5.2 probes the ability to capture locality patterns across pitch and time, which may enhance plausibility in generation. In Section 5.3, we study an additional real-time accompaniment task, validating our method's built-in compatibility to time-aligned control.

5.1. Analysis of Tokenization Compactness

We investigate compactness along two dimensions: *sequence length*, and the *proportion of compressible substructures*. A tokenization that derives shorter sequences allows for modeling longer musical contexts within a fixed context window. Meanwhile, a higher compression rate suggests the presence of more regular and reusable "substrings," which may help learn more generalizable structural patterns.

Our analysis on sequence length is shown in Table 3, where BEAT produces the most compact sequences. To assess compressibility, we use Byte Pair Encoding (BPE) compression rate as a proxy. Figure 4 illustrates the compression rates under an increasing number of BPE merges. BEAT consistently achieves lower compression rates (64.83% vs. 80.15% for Interleaved ABC and 80.18% for REMI at 20 merges), indicating that BEAT fosters a higher degree of reusable substructures. This regularity may further support the recognition and generalization of structural patterns.

5.2. Analysis of Structural Inductive Biases

We design three pattern-constrained generative tasks to evaluate locality-aware structural learning. For each task, we train models on synthetic data exhibiting a common pattern, using BEAT or REMI under the same architecture and hyperparameters. During generation, we measure how well the outputs capture the designated patterns. Higher pattern accuracy indicates stronger locality-aware representation learning and structural understanding, which are crucial for generating well-structured and plausible results.

All synthetic data consist of 8-bar segments in 4/4 time. We consider three types of constrained patterns: (1) **Stepwise**

Table 3. Average sequence length on the piano dataset.

Method	Avg. Length
Interleaved ABC	3450.4
REMI	1902.6
BEAT (Ours)	**1825.6**

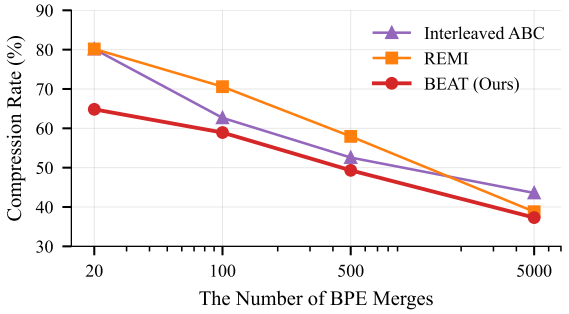


Figure 4. BPE compression rate across the number of BPE merges. Lower values indicate stronger regularity.

Transposition, where each bar is a one-semitone transposition of the previous bar, probing pitch-invariant locality; (2) **Beat Interleaving**, where each bar follows a fixed rhythmic pattern (AAAA, ABAB, or Mixed), evaluating beat-level structural regularities; (3) **Time-Shift Reconstruction**, where the first 4 bars are repeated after a delay of 0–3 beats, probing time-invariant locality.

For Stepwise Transposition and Beat Interleaving, we evaluate pattern accuracy at both the sequence and bar levels. Results are shown in Tables 4a and 4b, where BEAT consistently outperforms REMI in both cases. For Time-Shift Reconstruction, we evaluate note-level reconstruction accuracy, where generated outputs are converted into piano-roll representations, and frame-level precision is computed for onset and sustain states, respectively. As shown in Table 4c, BEAT achieves consistently high precision (93–97%), whereas REMI struggles particularly with 2-beat shifts (78%), which corresponds to a half-bar displacement that disrupts natural bar boundaries. These results demonstrate that, compared

Table 4. Probing task results comparing BEAT and REMI tokenization. Seq. = sequence-level accuracy (all 8 bars correct); Bar = per-bar accuracy. For time-shift reconstruction, we report frame-level precision on piano-roll onset and sustain states.

(a) Stepwise transposition accuracy (%)

Metric	REMI	BEAT	Δ
Seq.	28.28	**91.92**	+63.64
Bar	84.47	**98.48**	+14.01

(b) Beat interleaving accuracy (%)

Pattern	Metric	REMI	BEAT	Δ
AAAA	Seq.	49.48	**82.83**	+33.35
	Bar	89.18	**96.09**	+6.91
ABAB	Seq.	22.11	**76.29**	+54.18
	Bar	75.66	**93.94**	+18.28
Mixed	Seq.	7.37	**10.31**	+2.94
	Bar	60.39	**69.85**	+9.46

(c) Time-shift reconstruction precision (%)

Shift	REMI		BEAT	
	Onset	Sustain	Onset	Sustain
0 beat	88.3	89.8	**95.5**	**96.3**
1 beat	85.4	89.6	**95.0**	**97.0**
2 beats	78.0	79.3	**94.2**	**96.1**
3 beats	88.7	89.9	**93.9**	**95.9**

to event-based tokenization, BEAT learns stronger locality-aware representations that are robust to pitch and temporal displacement, preserving plausible and structured musical patterns. Full experimental setup for these probing tasks is provided in Appendix I.

5.3. Analysis of Real-Time Controllability

Under BEAT tokenization, a unified autoregressive framework can also be extended to real-time sequential control. In this section, we investigate the real-time accompaniment arrangement task, where the accompaniment token at time t is generated based on the melody context occurring strictly before t. This setting poses challenges to event-based tokenization, as their melody and accompaniment tokens are only asynchronously aligned (Thickstun et al., 2024). In contrast, under our formulation, melody and accompaniment are treated as parallel tracks composed of beat-level synchronized units. This allows them to be interleaved unit by unit within an autoregressive framework. Specifically, the ensuing model generates accompaniment for beat i conditioned on the past melody and accompaniment for beats $1, 2, \ldots, i-1$, ensuring strict causality and fine-grained temporal alignment.

We conduct experiment using melody-accompaniment pairs constructed from the piano test split described in Section 4.1.

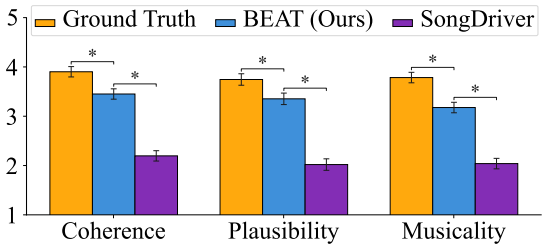


Figure 5. Subjective evaluation results for real-time accompaniment generation. Bar plots report mean ratings and standard errors. * indicates a statistically significant difference ($p < 0.05$).

We fine-tune our piano continuation model on the accompaniment generation task and compare it against **SongDriver** (Wang et al., 2022), a two-stage system specifically designed for real-time accompaniment. It first generates chord labels and then selects texture candidates in a rule-based manner. In comparison, our model takes an autoregressive approach in an end-to-end fashion.

We perform a subjective evaluation in the same setup as Section 4.4. Our survey consists of 5 pages, each presenting a 30-bar melody with an initial 4-bar accompaniment prompt. It is followed by accompaniment arrangements generated by our model and SongDriver, along with the ground-truth arrangement, all in random order. Participants evaluate each sample based on 3 criteria: 1) *Coherence* (accompaniment to melody), 2) *Plausibility*, and 3) *Musicality*. As shown in Figure 5, our method significantly outperforms SongDriver, producing more coherent and plausible musical accompaniments. This highlights our method's capacity for managing fine-grained, time-aligned sequence control even under real-time causal constraints.

6. Conclusion

To conclude, we contribute BEAT, a beat-granular tokenization for symbolic music that unifies advantages previously considered incompatible. While retaining compactness unique to event-based representations, BEAT explicitly models the temporal regularity inherent to grid-based piano-rolls, thereby preserving important musical priors such as pitch- and time-shift invariance. BEAT enables versatile control in music generation by unifying multiple tasks within a single autoregressive framework, including prompt-based continuation and real-time accompaniment generation. Extensive experiments demonstrate that BEAT produces coherent musical content with plausible long-term structure and enhanced musical quality. We hope that BEAT's principled design will bring new perspectives to future advances in symbolic music generation and understanding.

Impact Statement

This paper presents work aimed at advancing the field of generative music AI. Motivated by the success of large-scale lan-

guage models, we propose a structured tokenization method and evaluate its generative performance against existing representations such as MIDI-like, REMI, and ABC, as a step towards building foundation models for symbolic music. Our research has several positive impacts, particularly in enhancing artistic expression and creativity. Through our experiments, we demonstrate how users can transform an initial musical idea (whether a prompt or a melody) into a complete realization. The ensuing model can assist musicians and music learners in exploring brodaer creative choices, therby fostering an environment where innovation and artistic expression can thrive.

At the same time, we acknowledge the need to address potential risks. Increased accessibility to music generative models may lead to over-reliance on automation, potentially impeding the development of fundamental musical skills. We also recognize that our datasets predominantly reflect the Western musical tradition, which introduces a cultural bias that could limit the diversity of generated compositions. Widespread adoption of such models may lead to the homogenization of music, undermining the originality and individuality that are central to musical artistry.

References

Briot, J.-P., Hadjeres, G., and Pachet, F.-D. *Deep Learning Techniques for Music Generation*. Computational Synthesis and Creative Systems. Springer, 2019.

Chen, M., Radford, A., Child, R., Wu, J., Jun, H., Luan, D., and Sutskever, I. Generative pretraining from pixels. In *International Conference on Machine Learning*, pp. 1691–1703. PMLR, 2020.

Dong, H.-W., Chen, K., McAuley, J., and Berg-Kirkpatrick, T. MusPy: A toolkit for symbolic music generation. In *International Society for Music Information Retrieval Conference (ISMIR)*, pp. 142–149, 2020.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. URL https://arxiv.org/abs/2010.11929.

Esser, P., Rombach, R., and Ommer, B. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12873–12883, 2021.

Good, M. Musicxml: An internet-friendly format for sheet music. In *Proceedings of the XML Conference*. IDEAlliance, 2001.

Hsiao, W.-Y., Liu, J.-Y., Yeh, Y.-C., and Yang, Y.-H. Compound word transformer: Learning to compose full-song music over dynamic directed hypergraphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 178–186, 2021.

Huang, Y.-S. and Yang, Y.-H. Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions. In *Proceedings of the 28th ACM International Conference on Multimedia*, MM '20, pp. 1180–1188. Association for Computing Machinery, 2020. doi: 10.1145/3394171.3413671.

Huron, D. *Sweet Anticipation: Music and the Psychology of Expectation*. MIT Press, 2006.

Jeong, D., Kwon, T., Kim, Y., and Nam, J. Graph neural network for music score data and modeling expressive piano performance. In *International Conference on Machine Learning (ICML)*, pp. 3060–3070, 2019.

Jiang, J., Xia, G. G., Carlton, D. B., Anderson, C. N., and Miyakawa, R. H. Transformer vae: A hierarchical model for structure-aware and interpretable music representation learning. In *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 516–520, 2020a.

Jiang, N., Jin, S., Duan, Z., and Zhang, C. RL-duet: Online music accompaniment generation using deep reinforcement learning. In *AAAI Conference on Artificial Intelligence*, volume 34, pp. 710–718, 2020b.

Karystinaios, E. and Widmer, G. Cadence detection in symbolic classical music using graph neural networks. In *Proceedings of the 23rd International Society for Music Information Retrieval Conference*, Bengaluru, India, 2022.

London, J. Meter as a kind of attentional behavior. In *Hearing in Time: Psychological Aspects of Musical Meter*. Oxford University Press, 2nd edition, 2012. doi: 10.1093/acprof:oso/9780199744374.003.0001.

Lv, A., Tan, X., Lu, P., Ye, W., Zhang, S., Bian, J., and Yan, R. GETMusic: Generating any music tracks with a unified representation and diffusion framework. *arXiv preprint arXiv:2305.10841*, 2023.

MIDI Manufacturers Association. Midi 1.0 detailed specification. Technical report, MIDI Manufacturers Association, 1996.

Min, L., Jiang, J., Xia, G., and Zhao, J. Polyffusion: A diffusion model for polyphonic score generation with internal and external controls. In *Proceedings of the 24th International Society for Music Information Retrieval Conference (ISMIR)*, pp. 231–238, Milan, Italy, 2023.

Oore, S., Simon, I., Dieleman, S., Eck, D., and Simonyan, K. This time with feeling: learning expressive musical performance. *Neural Computing and Applications*, 32(4): 955–967, 2020.

Qu, X., Bai, Y., Ma, Y., Zhou, Z., Lo, K. M., Liu, J., Yuan, R., Min, L., Liu, X., Zhang, T., et al. MuPT: A generative symbolic music pretrained transformer. *arXiv preprint arXiv:2404.06393*, 2024.

Raffel, C. *Learning-based methods for comparing sequences, with applications to audio-to-MIDI alignment and matching*. PhD thesis, Columbia University, 2016.

Retkowski, J., Stepniak, J., and Modrzejewski, M. Frechet music distance: A metric for generative symbolic music evaluation. *arXiv preprint arXiv:2412.07948*, 2024.

Ryu, J., Dong, H.-W., Jung, J., and Jeong, D. Nested music transformer: Sequentially decoding compound tokens in symbolic music and audio generation. In *Proceedings of the 25th International Society for Music Information Retrieval Conference*, 2024.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Sun, P., Jiang, Y., Chen, S., Zhang, S., Peng, B., Luo, P., and Yuan, Z. Autoregressive model beats diffusion: Llama for scalable image generation. *arXiv preprint arXiv:2406.06525*, 2024.

Thickstun, J., Hall, D., Donahue, C., and Liang, P. Anticipatory music transformer. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=EBNJ33Fcrl.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

van den Oord, A., Vinyals, O., and Kavukcuoglu, K. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

von Rütte, D., Biggio, L., Kilcher, Y., and Hofmann, T. FIGARO: Controllable music generation using learned and expert features. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=NyR8OZFHw6i.

Walshaw, C. The abc music standard 2.1. https://abcnotation.com/wiki/abc:standard:v2.1, 2011. Accessed: 2025.

Wang, Y., Wu, S., Hu, J., Du, X., Peng, Y., Huang, Y., Fan, S., Li, X., Yu, F., and Sun, M. NotaGen: Advancing musicality in symbolic music generation with large language model training paradigms. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 10207–10215, 2025. doi: 10.24963/ijcai.2025/1134.

Wang, Z., Zhang, Y., Zhang, Y., Jiang, J., Yang, R., Xia, G., and Zhao, J. PIANOTREE VAE: Structured representation learning for polyphonic music. In *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR)*, pp. 368–375, 2020.

Wang, Z., Zhang, K., et al. SongDriver: Real-time music accompaniment generation without logical latency nor exposure bias. In *Proceedings of the 30th ACM International Conference on Multimedia*, pp. 1057–1067, 2022. doi: 10.1145/3503161.3548368.

Yang, R., Wang, D., Wang, Z., Chen, T., Jiang, J., and Xia, G. Deep music analogy via latent representation disentanglement. In *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, pp. 596–603, 2019.

Yuan, R., Lin, H., Wang, Y., Tian, Z., Wu, S., Shen, T., Zhang, G., Wu, Y., Liu, C., Zhou, Z., et al. Chatmusician: Understanding and generating music intrinsically with LLM. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 6252–6271, 2024. URL https://aclanthology.org/2024.findings-acl.373/.

A. Ablation Study

In this section, we conduct ablation studies to investigate the effect of two key design choices: (1) temporal granularity of patterns, and (2) pitch encoding strategy. All ablation experiments are performed on the piano continuation task using the piano dataset described in Section 4.1, with identical model architecture and training setup as detailed in Appendix D.

A.1. Temporal Granularity

The resolution parameter τ determines how many time steps (at sixteenth-note resolution) constitute a single beat unit. We compare three settings:

- $\tau = 2$ (0.5 beat): Each pattern token spans an eighth note. This finer granularity captures more temporal detail but results in longer sequences and smaller pattern vocabulary.
- $\tau = 4$ (1 beat, default): Each pattern token spans a quarter note. This aligns with the natural beat pulse in most Western music.
- $\tau = 8$ (2 beats): Each pattern token spans a half note. This coarser granularity produces shorter sequences but may lose important rhythmic details.

Table 5 presents the results. The default setting ($\tau = 4$) achieves the best performance across all metrics. Finer granularity ($\tau = 2$) increases sequence length without proportional benefit, while coarser granularity ($\tau = 8$) loses important rhythmic information, leading to degraded groove consistency.

Table 5. Ablation on temporal granularity (τ) for piano continuation.

Temporal Granularity	$JS_{GC}\downarrow$	$JS_{SC}\downarrow$	FMD$\downarrow$
$\tau = 2$ (0.5 beat)	0.060	0.040	464.4
$\tau = 4$ (1 beat, default)	**0.039**	**0.021**	**436.7**
$\tau = 8$ (2 beats)	0.145	0.086	438.2

A.2. Pitch Encoding Strategy

The default BEAT encoding sorts active pitches in descending order and encodes positions using relative intervals: the first pitch uses an absolute index ($d_1 = p_1$), while subsequent pitches use relative offsets ($d_j = p_{j-1} - p_j$ for $j \geq 2$). We compare against several alternative strategies:

- **Ascending + relative**: Sort pitches in ascending order; use relative intervals.
- **Descending + absolute**: Sort pitches in descending order; use absolute pitch indices for all positions ($d_j = p_j$ for all j).
- **Ascending + absolute**: Sort pitches in ascending order; use absolute pitch indices.
- **Random**: Randomly order active pitches within each beat; use absolute indices (relative encoding is not applicable without a consistent ordering).

Table 6 presents the results. The default strategy (descending + relative) achieves the best performance. Key observations:

- **Relative encoding outperforms absolute encoding**: Relative intervals provide transposition invariance—transposing a piece only affects the first pitch token d_1, while all subsequent intervals remain unchanged. This reduces the burden of learning key-specific patterns.
- **Descending order slightly outperforms ascending order**: In piano music, melodic content typically occupies higher registers while accompaniment patterns appear in lower registers. Descending order places the melodically salient pitches first, potentially facilitating coherent generation.

- **Deterministic ordering outperforms random ordering**: Consistent pitch ordering within beats provides a stable structure that the model can exploit, whereas random ordering introduces unnecessary variance.

Table 6. Ablation on pitch encoding strategy for piano continuation.

Pitch Encoding Strategy	$JS_{GC}\downarrow$	$JS_{SC}\downarrow$	FMD$\downarrow$
Descending + relative (default)	**0.039**	**0.021**	**436.7**
Ascending + relative	0.049	0.023	436.9
Descending + absolute	0.103	0.037	451.5
Ascending + absolute	0.119	0.035	452.6
Random	0.129	0.032	459.5

A.3. Summary

The ablation study validates two key design decisions in BEAT:

1. **Beat-level granularity ($\tau = 4$) is optimal**: This aligns with the natural beat pulse in music and provides a good balance between sequence compactness and temporal resolution.
2. **Relative pitch encoding with consistent ordering is beneficial**: This design choice leverages the transposition invariance property discussed in Section 3.2, reducing the learning burden and improving generalization across keys.

These results support the structural properties outlined in Section 3.2 and demonstrate that BEAT's design choices are well-motivated by both musical intuition and empirical performance.

B. Encoding and Decoding Algorithms

This section provides algorithmic details for the BEAT encoding and decoding procedures described in Section 3.1. We use consistent notation: $X \in \{0, 1, 2\}^{P \times T}$ denotes the piano-roll matrix, τ denotes steps per beat, and $N = \lceil T/\tau \rceil$ denotes the number of beats.

B.1. Single-Track Encoding

Algorithm 1 describes the encoding procedure for single-track music, implementing the three steps outlined in Section 3.1: (1) partitioning the piano-roll into beat segments $B^{(i)}$, (2) encoding each active pitch as a (pitch, pattern, velocity) tuple, and (3) assembling the token sequence with beat and bar markers.

B.2. Multi-Track Encoding

For multi-track music, we extend the single-track encoding with instrument tokens and bar-level interleaving. Algorithm 2 describes the procedure.

Track Ordering. Tracks are ordered by General MIDI program number in ascending order. This provides a consistent ordering across pieces: piano (0) appears before bass (32–39), which appears before strings (40–51), etc. Drum tracks (channel 10 in General MIDI) are placed last.

Instrument Prefix. Each track's content within a beat is prefixed by an instrument token `INS` k, where k corresponds to the MIDI program number. We group similar instruments into families (e.g., all piano variants map to a single token) to reduce vocabulary size while preserving instrument identity.

Bar-Level Interleaving. Within each bar, tracks are serialized sequentially. This design allows the model to observe the complete musical texture at each bar before proceeding, facilitating cross-track coherence.

B.3. Decoding

Decoding reconstructs the piano-roll matrix X from a BEAT token sequence. The key insight enabling efficient decoding is that beat boundaries are implicitly marked: the first pitch token d_1 in each beat uses an absolute index (non-negative), while

Algorithm 1 BEAT Encoding (Single-Track)

Require: piano-roll $X \in \{0, 1, 2\}^{P \times T}$, velocity matrix $V \in \mathbb{R}^{P \times T}$, steps per beat τ, beats per bar n
Ensure: Token sequence $\mathbf{E}$

$N \leftarrow \lceil T/\tau \rceil$ {Number of beats}
$\mathbf{E} \leftarrow [\,]$
for $i = 1$ to N **do**
  if $(i-1) \mod n = 0$ **then**
    Append `BAR` to $\mathbf{E}$
  end if
  Append `BEAT` to $\mathbf{E}$
  $B^{(i)} \leftarrow X[:, (i-1)\tau : i\tau]$ {Beat segment}
  $\mathcal{A} \leftarrow \{p : \exists\, t,\ B^{(i)}[p, t] \neq 0\}$ {Active pitches}
  if $\mathcal{A} = \emptyset$ **then**
    Append `REST` to $\mathbf{E}$ and continue
  end if
  Sort $\mathcal{A}$ descending: $p_1 > p_2 > \cdots > p_M$ where $M = |\mathcal{A}|$
  for $j = 1$ to M **do**
    $\mathbf{s}_{p_j} \leftarrow B^{(i)}[p_j, :]$ {State vector}
    $s_{p_j} \leftarrow \textsc{Base3ToInt}(\mathbf{s}_{p_j})$ {Pattern token}
    $v_{p_j} \leftarrow \textsc{MeanVelocity}(V[p_j, (i-1)\tau : i\tau])$ {Velocity token}
    if $j = 1$ **then**
      $d_1 \leftarrow p_1$ {Absolute pitch}
    else
      $d_j \leftarrow p_{j-1} - p_j$ {Relative interval}
    end if
    Append $(\texttt{PIT}\, d_j,\ \texttt{PAT}\, s_{p_j},\ \texttt{VEL}\, v_{p_j})$ to $\mathbf{E}$
  end for
end for
return $\mathbf{E}$

subsequent pitch tokens d_j for $j \geq 2$ use relative intervals (negative). Algorithm 3 describes the procedure.

Invalid Token Handling. During autoregressive generation, the model may produce invalid tokens. We apply the following strategies:

- **Out-of-range pitch**: If the accumulated pitch p falls outside $[0, P-1]$, skip the token and continue decoding. This preserves temporal alignment while discarding the invalid note.
- **Invalid pattern**: If $s \notin [0, 3^\tau - 1]$, treat as a rest pattern (all zeros). With $\tau = 4$, valid patterns are $[0, 80]$.
- **Unexpected token type**: If a token appears in an invalid context (e.g., `BAR` within a beat), skip it and continue.

These strategies ensure robust decoding even when the model produces occasional errors, maintaining temporal structure while gracefully handling anomalies.

C. Dataset Details

C.1. Data Sources

Our training corpus combines two complementary sources:

- **Lakh MIDI Dataset** (Raffel, 2016): 176,581 MIDI files scraped from the web, representing a broad distribution of popular and classical music with varying quality.

Algorithm 2 BEAT Encoding (Multi-Track)

Require: Track piano-rolls $\{X_k\}_{k=1}^{K}$, instrument programs $\{I_k\}_{k=1}^{K}$, steps per beat τ, beats per bar n
Ensure: Token sequence $\mathbf{E}$
Sort tracks by program number: $I_{\pi(1)} \leq I_{\pi(2)} \leq \cdots \leq I_{\pi(K)}$
$N \leftarrow \lceil T/\tau \rceil, \quad N_{\text{bars}} \leftarrow \lceil N/n \rceil$
$\mathbf{E} \leftarrow [\,]$
for $m = 1$ to N_{bars} **do**
  Append `BAR` to $\mathbf{E}$
  for $k = 1$ to K **do**
    Append `INS` $I_{\pi(k)}$ to $\mathbf{E}$
    for $i = (m-1)n + 1$ to $\min(mn, N)$ **do**
      Append `BEAT` to $\mathbf{E}$
      $\mathbf{u}_{\pi(k)}^{(i)} \leftarrow \textsc{EncodeBeat}(X_{\pi(k)}, i, \tau)$
      Append $\mathbf{u}_{\pi(k)}^{(i)}$ to $\mathbf{E}$
    end for
  end for
end for
return $\mathbf{E}$

- **MuseScore Collection**: 1,652,471 user-contributed scores in MuseScore format (.mscz), collected following the methodology described at `https://github.com/Xmader/musescore-dataset`. This collection provides higher-quality notation data with cleaner beat quantization and explicit track annotations.

The MuseScore format offers several advantages over raw MIDI: explicit beat boundaries, cleaner quantization, and richer metadata. We convert all files to a unified MIDI representation for training.

C.2. Filtering Criteria

We filter for quality: retain pieces with 8–200 bars, remove non-standard instruments, filter quantization anomalies, deduplicate by content hash, and remove empty tracks.

C.3. Dataset Statistics

Table 7 summarizes the dataset statistics after filtering.

Table 7. Dataset statistics after preprocessing.

Subset	Lakh	MuseScore	Total
Multi-track (filtered)	148,056	–	148,056
Piano (filtered)	15,157	192,789	207,946

C.4. Piano Subset Construction

For accompaniment experiments, we extract pieces with exactly two piano tracks (melody and accompaniment). The melody serves as conditional input and the accompaniment as generation target.

D. Training Details

Table 8 summarizes the model architecture used in all experiments. We adopt a decoder-only Transformer following LLaMA (Touvron et al., 2023) with Rotary Position Embedding (RoPE) (Su et al., 2024).

Table 9 lists the training hyperparameters.

Batch Construction. Sequences are packed to maximize GPU utilization. We concatenate multiple pieces with [EOS]

Algorithm 3 BEAT Decoding (Single-Track)

Require: Token sequence $\mathbf{E}$, steps per beat τ
Ensure: piano-roll $X \in \{0, 1, 2\}^{P \times T}$

Initialize $X \leftarrow \mathbf{0}^{P \times T}$
$i \leftarrow 0, \quad p_{\text{prev}} \leftarrow \textsc{None}$
for each token in $\mathbf{E}$ **do**
  if token is `BAR` **then**
    Continue
  else if token is `BEAT` **then**
    $i \leftarrow i + 1, \quad p_{\text{prev}} \leftarrow \textsc{None}$
  else if token is `REST` **then**
    Continue {Beat already incremented}
  else if token is (`PIT` d, `PAT` s, `VEL` v) **then**
    if $d \geq 0$ **then**
      $p \leftarrow d$ {Absolute pitch (anchor)}
    else
      $p \leftarrow p_{\text{prev}} + d$ {Relative pitch}
    end if
    if $0 \leq p < P$ **then**
      $X[p, (i-1)\tau : i\tau] \leftarrow \textsc{IntToBase3}(s, \tau)$
    end if
    $p_{\text{prev}} \leftarrow p$
  end if
end for
return X

tokens as separators until reaching the context length of 2048. Sequences exceeding this length are truncated.

Checkpoint Selection. We evaluate validation loss every epoch and select the checkpoint with the lowest validation loss for final evaluation.

E. Baselines

For fair comparison, all baseline methods use identical model architecture (Section D) and training data, differing only in tokenization.

E.1. REMI / REMI+

We implement REMI following Huang & Yang (2020). The token vocabulary includes:

- **Bar**: Bar boundary marker
- **Position**: Position within bar (0–15 for 16th-note resolution)
- **Pitch**: MIDI pitch number (0–127)
- **Duration**: Note duration in 16th notes
- **Velocity**: Quantized velocity (we use 32 bins)

REMI+ extends REMI with **Track** tokens for multi-track music, following von Rütte et al. (2023). For single-track piano, REMI+ reduces to standard REMI.

Table 8. Model architecture hyperparameters.

Hyperparameter	Value
Number of layers	16
Hidden dimension	768
Number of attention heads	12
Feed-forward dimension	3072
Dropout rate	0.1
Context length	2048
Positional encoding	RoPE
RoPE base frequency	10000
Total parameters	∼150M

Table 9. Training hyperparameters.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1×10^{-4}
β_1, β_2	0.9, 0.999
Weight decay	0.01
Batch size	256
Training epochs	30
LR schedule	Cosine with linear warmup (1 epochs)
Hardware	4× RTX 3090
Training time	∼240 GPU hours(30 epochs)

E.2. Compound Word

We implement Compound Word following Hsiao et al. (2021). Each compound token packages multiple attributes:

$$\text{Token} = (\text{Type}, \text{Pitch}, \text{Duration}, \text{Velocity}) \tag{5}$$

The model predicts all attributes simultaneously, reducing sequence length compared to REMI. We use the same attribute vocabulary as REMI.

E.3. Interleaved ABC

We implement multi-track ABC notation following Wang et al. (2025). Key features:

- Character-level tokenization of ABC notation
- Voice headers (V:1, V:2, etc.) for track separation
- Measure bars (—) for structural alignment

E.4. Task-Specific Baselines

Anticipatory Music Transformer (Thickstun et al., 2024): An autoregressive model with anticipation mechanism for infilling. We use the official released checkpoint.

SongDriver (Wang et al., 2022): A two-stage model for real-time accompaniment. We use the official released model and follow their evaluation protocol.

Note: These baselines differ from BEAT in architecture, model size, and training data. Comparisons should be interpreted as evaluating overall system performance rather than isolated representation effects.

F. Objective Evaluation Metrics

We adopt metrics from MusPy (Dong et al., 2020) and measure distributional similarity via Jensen-Shannon divergence.

Groove Consistency (GC). Groove consistency measures rhythmic regularity across measures (Dong et al., 2020). For each piece, we compute the mean similarity of onset patterns between adjacent measures:

$$\text{GC} = 1 - \frac{1}{T-1}\sum_{i=1}^{T-1} d(\mathbf{g}_i, \mathbf{g}_{i+1}) \tag{6}$$

where T is the number of measures, $\mathbf{g}_i \in \{0,1\}^R$ is the binary onset vector of measure i (with R time steps per measure), and $d(\cdot,\cdot)$ is the normalized Hamming distance. Higher values indicate more consistent rhythmic patterns across measures.

Scale Consistency (SC). Scale consistency measures tonal coherence (Dong et al., 2020). For each piece, we compute the maximum pitch-in-scale rate across all major and minor scales:

$$\text{SC} = \max_{\text{root,mode}} \frac{|\{p : p \in \text{Scale}(\text{root}, \text{mode})\}|}{|\{p\}|} \tag{7}$$

where p denotes pitch classes of all notes, and Scale(root, mode) defines the pitch classes belonging to the specified scale. Higher values indicate better adherence to a consistent tonal center.

Fréchet Music Distance (FMD). Following Retkowski et al. (2024), we compute FMD using CLaMP2 embeddings:

$$\text{FMD} = \|\mu_g - \mu_r\|^2 + \text{Tr}(\Sigma_g + \Sigma_r - 2(\Sigma_g\Sigma_r)^{1/2}) \tag{8}$$

where (μ_g, Σ_g) and (μ_r, Σ_r) are the mean and covariance of generated and reference embeddings, respectively.

G. Subjective Evaluation Details

Our subjective evaluation is conducted via an online crowdsourcing study, where participants complete a survey consisting of listening and rating tasks. This section provides additional details on the survey design and participant profile.

G.1. General Instructions

Participants receive the following general instructions, which clarify their rights and the conditions of participation:

- Participation is entirely voluntary, and one may withdraw at any time without any negative consequences.
- No personally identifying information is collected; all responses are anonymous and used solely for research purposes.

G.2. Survey Design

Our survey consists of 5 pages for *piano continuation*, *multi-track continuation*, and *real-time accompaniment*, respectively (15 pages in total). Each page presents outputs from different models corresponding to a common test input, which are MIDI prompts or melodies drawn from the respective test split. Outputs are generated by our method and all baselines, with the ground-truth sample included as a perceptual anchor. All models to be evaluated, along with the ground truth, are anonymised, and the presentation order on each page is randomized. We distribute our survey via SurveyMonkey.[2]

Participants listen to each sample and evaluate its musical quality on a 5-point Likert scale from 1 to 5. The evaluation considers 3 criteria:

- **Coherence**: How well the continuation/accompaniment aligns with the prompt/melody.
- **Plausibility**: How musically valid and well-formed the continuation/accompaniment is (considering longer-term music structure).

[2] https://www.surveymonkey.com/

- **Musicality**: The overall musical quality.

To analyze the data, we first tested the normality of response distributions, and then conducted within-subject ANOVA followed by post-hoc pairwise t-tests to assess statistical significance. This ensures that observed differences in ratings reflect differences among the models rather than individual participant variability. To maintain response quality, we only included complete evaluation sets, requiring participants to rate all models on a given page for their responses to be considered valid.

G.3. Completion Time

Each participant is randomly assigned 5 out of the 15 pages. On each page, participants first listen to the test input piece, followed by the corresponding output samples, which they then rate. All samples in the survey are 30 bars long and rendered to audio using the MuseScore[3] soundfonts, producing approximately 1min of audio per sample. This design targets a total completion time of 25 minutes, ensuring that participants have sufficient time to listen carefully without excessive fatigue. The actual completion time observed on average is 32min 25s.

G.4. Participant Profiles

Participants are asked to self-identify their musical background as *amateur*, *intermediate*, or *professional*, following the guidelines below:

- **Amateur**: I enjoy listening to music. I can play/sing/compose short music pieces. I know a little music theory. I can evaluate a composition based on my feelings.
- **Intermediate**: I have some experience in performing, composing, or other music activities. I know a certain amount of music theories that can help me evaluate a composition.
- **Professional**: I am now pursuing/have completed a music degree, or having equivalent background. I am proficient in using music theory to evaluate a composition.

Among the 32 valid responses, 12 participants identified as *amateur* (37.5%), 10 as *intermediate* (31.25%), and 10 as *professional* (31.25%). All authors are excluded from the survey.

H. Qualitative Examples

Generated examples and interactive demonstrations are available on our demo page: https://anonymous.4open.science/w/BEAT-349F/.

I. Probing Experiment Details

This section provides additional details for the structural pattern learning experiments in Section 5.2. All experiments use the same model architecture as Table 8. BEAT and event-based baselines are trained with identical hyperparameters, differing only in tokenization.

Dataset Split. All three tasks use 4,000 samples split 9:1 into training (3,600) and validation (400) sets. Source material is derived from 4/4 time music in the piano dataset.

I.1. Stepwise Transposition

Dataset. We construct synthetic 8-bar sequences where each bar is a one-semitone transposition of the previous bar (Figure 6). This pattern probes pitch-invariant locality.

Evaluation. Models generate 200 sequences unconditionally (temperature=1.0, top-p=0.95). We report:

- *Sequence Accuracy*: percentage of sequences where all 8 bars follow the target transposition pattern.
- *Bar Accuracy*: percentage of individual bars matching the expected transposition.

[3] https://musescore.org/

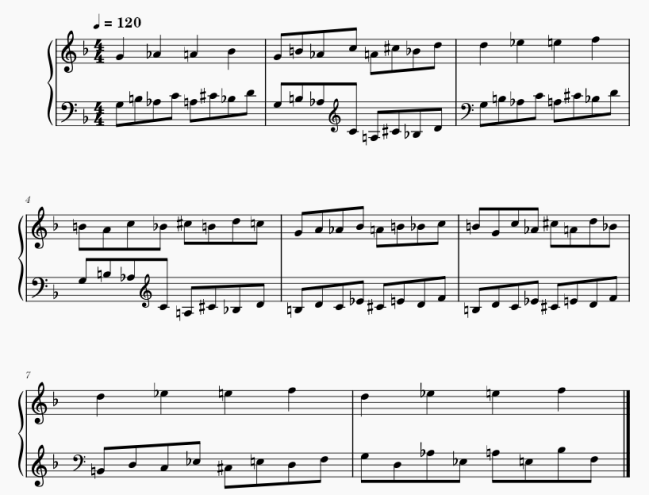


Figure 6. Stepwise transposition pattern: each bar is transposed up by one semitone from the previous bar.

I.2. Beat Interleaving

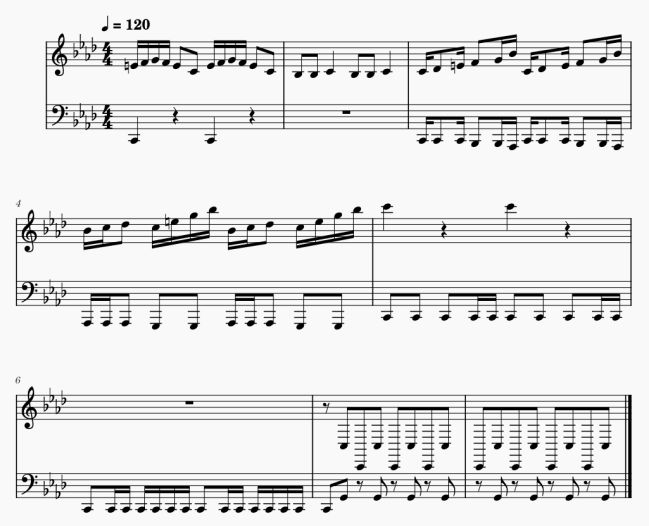


Figure 7. ABAB pattern: beats alternate in an A-B-A-B pattern within each bar.

Dataset. We construct synthetic 8-bar sequences with controlled rhythmic structures. Each bar contains 4 beats following a specific pattern:

- **AAAA** (Figure 8): Four identical beats per bar. Beat content may differ across bars while the structural pattern remains consistent.
- **ABAB** (Figure 7): Beats alternate in an A-B-A-B pattern within each bar.
- **Mixed** : AAAA and ABAB alternate across segments, testing higher-order pattern learning.

This task evaluates whether models can learn beat-level structural regularities.

Evaluation. Models generate 200 sequences unconditionally (temperature=1.0, top-p=0.95). We report Sequence Accuracy and Bar Accuracy as defined above.

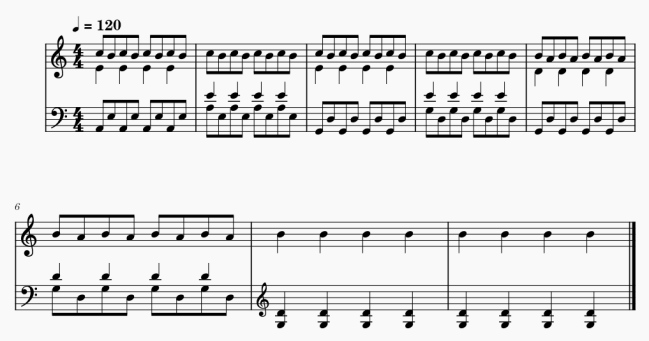


Figure 8. AAAA pattern: four identical beats per bar.

I.3. Time-Shift Reconstruction

Dataset. Each sequence contains a 4-bar prompt followed by the same content delayed by $k \in \{0, 1, 2, 3\}$ beats. This pattern probes time-invariant locality.

Evaluation. We use a held-out set of 400 samples per shift amount for final evaluation. Given the 4-bar prompt, models generate the time-shifted continuation using deterministic decoding (top-k=1). We convert outputs to piano-roll and compute frame-level precision for onset and sustain states separately, measuring note-level reconstruction fidelity.